From Digital Systems to Microcontrollers

Introduction to ARM Cortex-M Microprocessors — Chapter 1

Muním Zabidi

Faculty of Electrical Engineering, Universiti Teknologi Malaysia

Table of Contents

1. Why This Matters
2. Embedded Systems
3. Microprocessor vs Microcontroller
4. Buses and Memory-Mapped I/O
5. Inside the CPU
6. Execution Cycle and Program Trace
7. Meet the Board
8. From C to a Pin

Why This Chapter Matters

Your gate circuits did only what their wiring told them. To change the behaviour, you had to rewire. A microcontroller is a digital circuit too, but a program stored in memory controls it. The same silicon can blink an LED, measure a temperature, or drive a motor. It depends only on the bits you load.

After this chapter, you can:

1. Define an embedded system and its design constraints.
2. Distinguish a microprocessor from a microcontroller.
3. Name the CPU functional units and the role of each.
4. Explain the address, data and control buses, and compute addressable memory.
5. Trace a short program through fetch–decode–execute.
6. Describe how one C statement changes a voltage on a pin.

© Munim Zabidi

This chapter introduces many new terms. Every section answers just one question. This question is the thread that connects everything that follows:

> How does a program cause hardware to do something?

The one fact to carry out of this chapter

The CPU has no special instruction for “turn on an LED”.

It only has instructions that read and write memory addresses. Some of these addresses connect not to memory, but to hardware instead.

So turning on an LED means writing a number to an address.

© Munim Zabidi

Before we learn the vocabulary, let us see where we are going. One line of C changes a physical pin:

```
GPIOA->ODR |= (1U << 5);
```



By the end of this chapter you will be able to explain every arrow in this chain. You do not need to understand it yet: this is our roadmap.

© Munim Zabidi

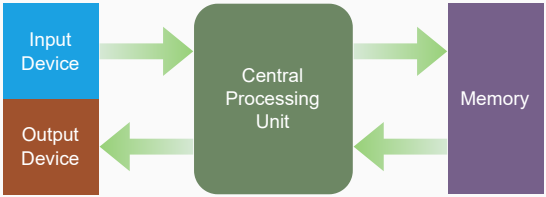
Table of Contents

1. Why This Matters
2. Embedded Systems
3. Microprocessor vs Microcontroller
4. Buses and Memory-Mapped I/O
5. Inside the CPU
6. Execution Cycle and Program Trace
7. Meet the Board
8. From C to a Pin

Components of the Computer

- Since 1946 all computers have had 4 components:

Processor	controls the system
Memory	contains program and data
Input	devices that get data from the outside world into the computer
Output	devices that present computation results to the outside world



Together they perform four functions: data processing, storage, movement, and control.

What Is the CPU?

- The CPU is the brain of the computer. It works by executing a program.
- **Program**: a sequence of instructions that performs a task.
  - Think of running a program like playing music.
- **Instruction**: a binary code for the simplest operation the processor can perform.
  - Think of each instruction as one note.

- Memory is a $k \times m$ array of stored bits (k is usually 2^n).
  - **Address** unique n-bit identifier of a location
  - **Contents** m-bit value stored in that location
- Two basic operations:
  - **Read** get the value from a memory location
  - **Write** put a value into a memory location

© Muním Zabidi

Memory: ROM and RAM

ROM — non-volatile

- Contents survive with no power
- Holds the program: fixed at manufacture, or "burned" with a programmer
- **Flash** is the dominant type: NAND or NOR
- STM32 devices use NOR flash for code

RAM — volatile

- Contents lost without power
- Holds temporary data: variables, the stack
- **SRAM**: fast, every embedded system has it
- **DRAM**: cheaper, needs refresh; only larger systems bother

On the Nucleo-F446RE

512 KB Flash holds your program. **128 KB SRAM** holds your variables and stack. Chapter 3 returns to this split in detail.

© Muním Zabidi

What Makes a System Embedded

Every computer, from server to thermostat, is processor, memory, and I/O. Embedded systems outnumber every other kind combined: dozens per person, hidden inside everyday products. The difference is what the processor is for. A general-purpose computer is built to run whatever the user installs next. An embedded system is built into a product to perform one job.

General-purpose computer

- Runs many different applications
- End-user installs new software freely
- Faster is always better
- Sold on speed, compatibility, price

Embedded system

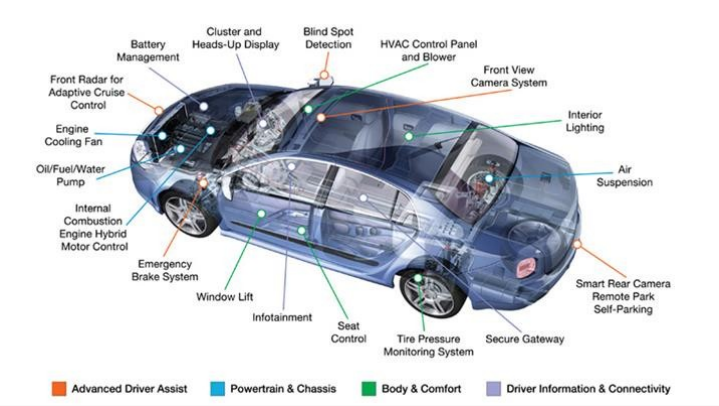
- Runs a few functions, fixed at design time
- Firmware is written once, by the designer
- Must meet a fixed deadline; extra speed may not help
- Sold on power, cost, size, predictability

The idea that matters

A desktop that answers in 100 ms instead of 10 ms is just slow. But an airbag controller that fires in 100 ms instead of 10 ms has failed.

Embedded engineering cares about *predictable* time, not maximum speed.

Embedded Systems: An Example



Each label is at least one ECU: an **Electronic Control Unit**. It is a micro-controller, its firmware, and a network connection, all together.

A car today carries **50–150** of these. By **2030**, everyday cars are expected to carry 200–250.

The driver never programs any of them directly.

Diagram credit: Embitel.

An ECU is classified by the function it controls: engine control module (ECM), body control module (BCM), electronic brake control module (EBCM), powertrain control module (PCM), transmission control module (TCM), suspension control module (SCM), door control unit (DCM), battery management system (BMS), and many more.

© Muním Zabidi

Table of Contents

1 Why This Matters

2 Embedded Systems

3 **Microprocessor vs Microcontroller**

4 Buses and Memory-Mapped I/O

5 Inside the CPU

6 Execution Cycle and Program Trace

7 Meet the Board

8 From C to a Pin

Constraint	Why it binds	What it forces
Size	Fits inside the product it controls	Package, pin count, board area
Weight	Portable, airborne, wearable	Battery size, materials
and		
Power	Battery life, thermal budget	Clock speed, sleep modes, duty cycle
Cost	Multiplied by production volume	Part choice, external components

Cost is multiplied, not added

Saving RM0.50 on one part looks trivial. But at 500,000 units, that saving becomes RM250,000. This is why embedded engineers argue about single resistors.

Apply it: why might a washing machine use a microcontroller rather than a general-purpose PC?

© Muním Zabidi

SWaP-C is a design constraint. Integrating the processor, memory, and peripherals onto one chip is how that constraint gets met in practice: fewer parts, smaller board, lower cost per unit. This single-chip design is called a **microcontroller**.

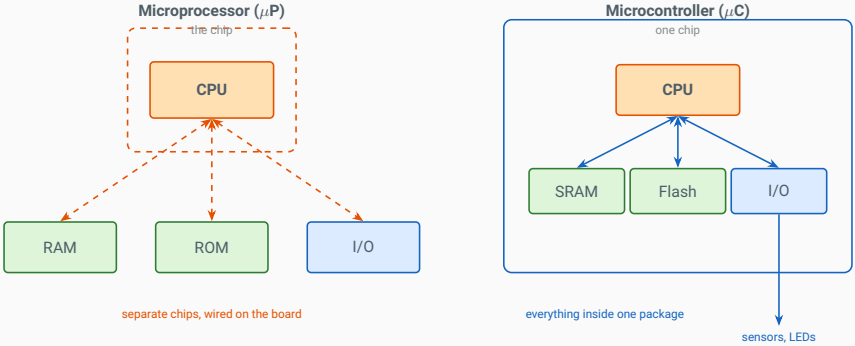
Feature	Microprocessor (μP)	Microcontroller (μC)
Integration	Processor chip only	Processor, RAM, ROM, I/O on one die
Flexibility	Memory and peripherals scale	Resources fixed inside the chip
Complexity	Many components, complex PCB	Few components, simple board
Power, cost	Higher	Optimised for low power and unit cost
Performance	High clock, heavy computation	Lower clock, control and I/O

A microprocessor needs help; a microcontroller comes ready

A microprocessor generally has no RAM, no ROM, and no way to reach the outside world on-chip; those parts must be wired around it before it can do anything. A microcontroller puts all of this together on one chip. The Nucleo's chip is a microcontroller: everything on the earlier board slide sits inside one package.

© Muním Zabidi

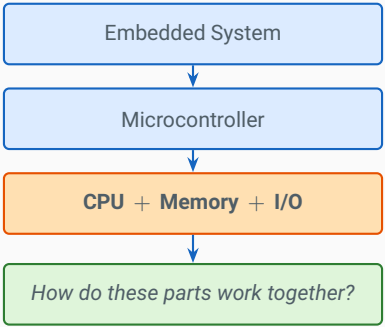
Where the Chip Boundary Falls



A microprocessor needs memory and I/O added around it. A microcontroller brings them on the chip: fewer parts, a smaller board, a lower cost.

Every Embedded System Is Still a Computer

To understand how firmware controls hardware, we must first understand how a computer moves and processes information internally.



The next sections open the microcontroller: buses, memory, and the CPU itself.

© Muním Zabidi

Zooming In: From System to Operation

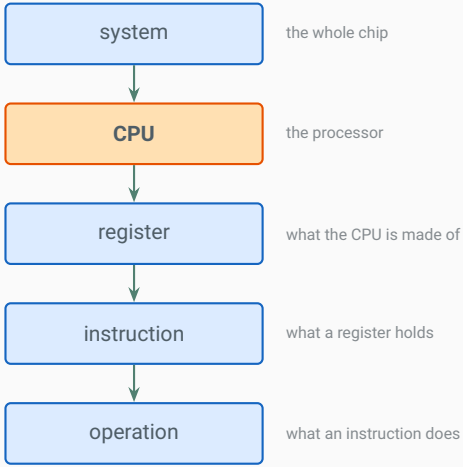
The next few slides introduce several names together: buses, registers, ALU, decoder. Each one belongs to one of two jobs: **moving** data from place to place, or **processing** it once it arrives.

Think of this as one picture, zoomed in one level at a time — top to bottom in the chain on the right.

Level 1: the whole microcontroller

CPU, RAM, Flash, GPIO, UART, ADC — all on one chip, connected by buses.

Buses move data; the CPU's internal parts process it. Buses let the CPU *reach* the rest of the chip. Registers are what the CPU is *made of*.



© Munim Zabidi

Table of Contents

1 Why This Matters

2 Embedded Systems

3 Microprocessor vs Microcontroller

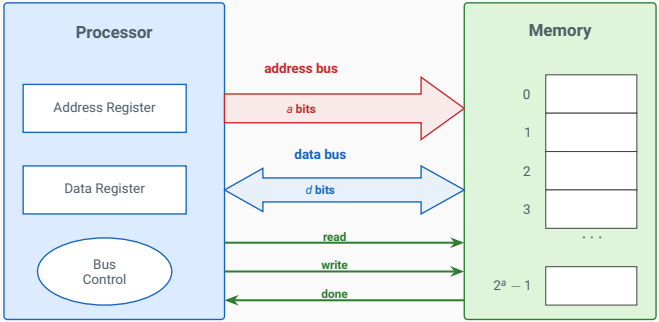
4 **Buses and Memory-Mapped I/O**

5 Inside the CPU

6 Execution Cycle and Program Trace

7 Meet the Board

8 From C to a Pin



Address Bus
Which location.
One-way (CPU → Mem).
a bits → 2^a locations.

Data Bus
What value.
Two-way.
d bits per transfer.

Control Bus
When & How.
Read, Write, Timing.
Coordinates the transfer.

You already know this: an address feeds a decoder, like your 3-to-8 decoder, just at a larger scale. *Note: The Cortex-M4 uses a bus matrix with several physical buses, but this 3-bus model captures the core idea.*

© Muním Zabidi

- The address bus carries the address of the memory location or I/O device selected for a transfer.
- Its width sets the addressing range: an n-bit address bus reaches 2^n locations.

CPU	Address bus width	Addressable memory
8051	16 bits	2^{16} = 65,536 bytes = 64 KB
8086	20 bits	2^{20} = 1,048,576 bytes = 1 MB
68000	24 bits	2^{24} = 16,777,216 bytes = 16 MB
ARM Cortex-M4	32 bits	2^{32} = 4,294,967,296 bytes = 4 GB

© Muním Zabidi

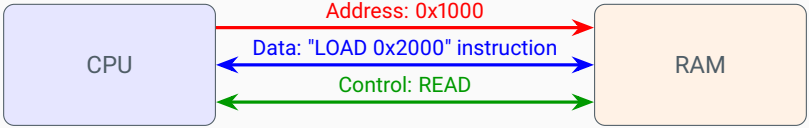
- The width of the data bus sets how much data moves in one transfer.
- Classic 8-bit microcontrollers have an 8-bit data bus.
  - It moves 1 byte at a time.
  - A 32-bit word therefore takes 4 transfers.
- The ARM Cortex-M4 has a 32-bit data bus.
  - It moves 4 bytes at once, so a 32-bit word takes 1 transfer.
- Some chips offer an external bus of selectable width (8, 16 or 32 bits).
  - A narrower bus is slower, but lets you use cheaper memory devices.

© Muním Zabidi

Using the Buses: Fetching an Instruction

Suppose the CPU needs the instruction stored at `0x1000`:

Bus	Carries, this step
Address bus	`0x1000` — CPU asks for this location
Control bus	`READ` — CPU asserts a read
Memory	responds with `LOAD 0x2000`
Data bus	carries that instruction back to the CPU



The same pattern repeats for every data access

The address bus names the location. The control bus says read or write. The data bus carries the value. Three buses, one transaction. We will use this exact pattern again when we trace a program.

© Munim Zabidi

Memory Organization: Addresses and Contents

Memory is a list of numbered cells. Each cell has an address and holds a value.

Address	Contents
0x0000	3A
0x0001	F2
0x0002	17
0x0003	90
...	...

- The address bus selects one row.
- The data bus carries the contents of that row.
- With n address bits, memory has 2^n rows; each row is m bits wide.

The addresses are the same, whether the row holds RAM, Flash or a peripheral register

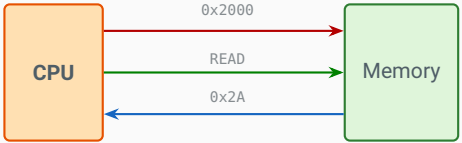
That is the single fact that makes memory-mapped I/O work.

© Muním Zabidi

Accessing Memory: Read versus Write

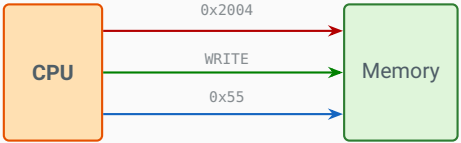
Every access is the same three steps. Only the direction of the data differs.

Read: memory → CPU



1. CPU puts the address `0x2000` on the address bus.
2. CPU asserts the READ signal on the control bus.
3. Memory responds with the data `0x2A` on the data bus.

Write: CPU → memory



1. CPU puts the address `0x2004` on the address bus.
2. CPU asserts the WRITE signal on the control bus.
3. CPU puts the data `0x55` on the data bus; memory stores it.

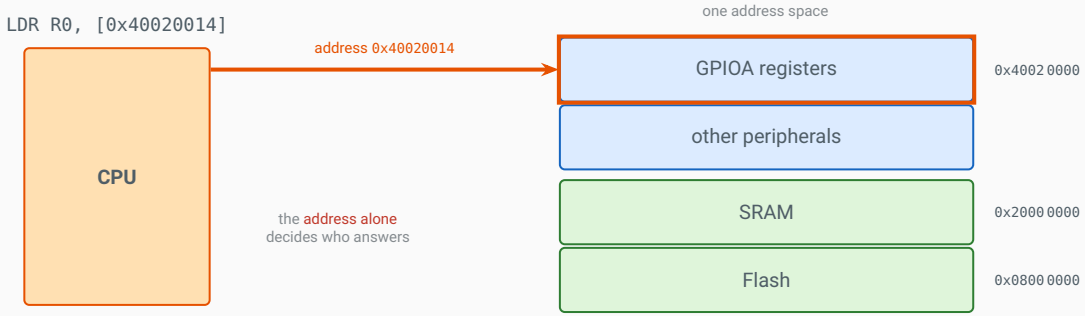
Same address, same control, opposite data: on a read the memory drives the data bus; on a write the CPU does.

Check yourself: the CPU puts `0x2000` on the address bus and requests a read. Which bus carries the stored value back?

© Muním Zabidi

Memory-Mapped I/O

The CPU reaches peripherals the same way it reaches memory: one address space, one bus, the same LDR/STR.



Key idea: each peripheral register has an address in the memory map. A read or write there reaches the register, not RAM — the peripheral answers. This is the exact mechanism that will turn on the LED from slide 5.

Check yourself: what determines whether a read reaches SRAM or a peripheral register?

© Munim Zabidi

Same Transaction, Different Destination

The CPU uses the same instruction for RAM and for a peripheral. Only the address differs.



`LDR R0, [R1]` returns the stored data.

Consequence: a value in memory moves into a register.

`STR R0, [R1]` drives a pin.

Consequence: the LED from our roadmap turns on.

Same bus transaction. Different destination. Different consequence.

Peripherals need no special instructions — the address alone selects what the `LDR`/`STR` reaches.

© Muním Zabidi

Table of Contents

1 Why This Matters

2 Embedded Systems

3 Microprocessor vs Microcontroller

4 Buses and Memory-Mapped I/O

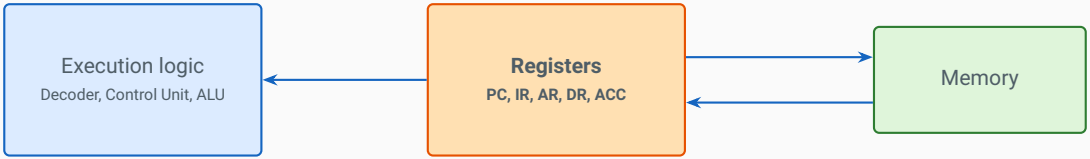
5 **Inside the CPU**

6 Execution Cycle and Program Trace

7 Meet the Board

8 From C to a Pin

The buses are highways; the CPU is a factory. It has a receiving dock (registers that face memory) and an assembly line (decoder, control unit, ALU).

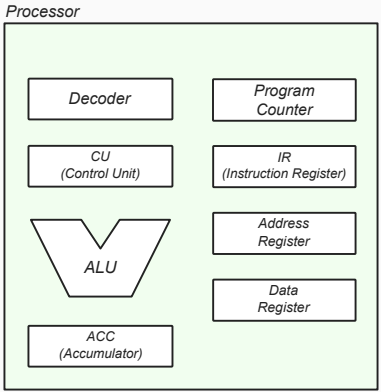


The rule that shapes everything else

In this generic teaching CPU, only the registers touch memory. The decoder, control unit, and ALU have no direct memory access of their own — everything they need must first be fetched into a register. Chapter 2 introduces the real Cortex-M4 registers.

The CPU Faces Memory: PC, IR, AR, DR

Four registers sit between the CPU and memory. Each one holds something that is going to or coming from memory.



Memory

0x1000	LOAD 0x2000
0x1002	ADD 0x2002
0x1004	STORE 0x2004
0x1006	.....
.....	.....
.....	.....
0x2000	1
0x2002	2
0x2004	?

PC Program Counter: the address of the next instruction.

IR Instruction Register: the instruction just fetched.

AR Address Register: the address of the data being accessed.

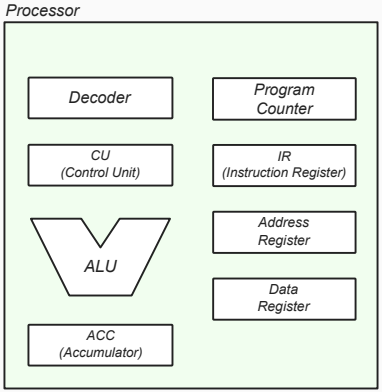
DR Data Register: the value in transit to or from that address.

Only these registers talk to memory. Everything else in the CPU works on what they hold.

© Muním Zabidi

The CPU Computes: Decoder, CU, ALU, ACC

The rest of the CPU does the work on what the registers hold.



Memory

0x1000	LOAD 0x2000
0x1002	ADD 0x2002
0x1004	STORE 0x2004
0x1006	.....
.....	.....
.....	.....
0x2000	1
0x2002	2
0x2004	?

Decoder Reads the bit pattern in IR and works out what operation and which operands it names.

CU Control Unit: generates the control signals: which register reads, which writes, when the ALU runs.

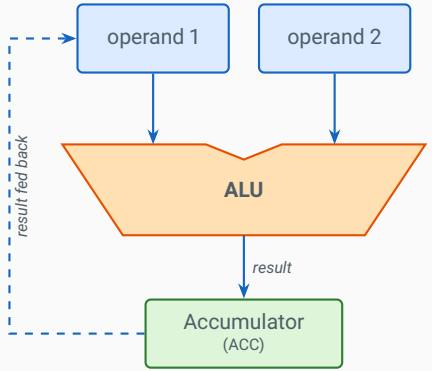
ALU Arithmetic Logic Unit: add, subtract, AND, OR, XOR.

ACC Accumulator: operand and result of the ALU.

The ALU's bit width is "the size of the processor": 4 bits for a remote control, 8 for classic microcontrollers (8051, PIC), 32 for the Cortex-M4, 64 for servers.

© Muním Zabidi

The Accumulator: Where Results Live



- The **ACC** is a key general-purpose register.
- Before an ALU operation, it holds an operand.
- After the operation, it receives the result.
- The dashed arrow shows why this matters: a result can become the next operand, without ever leaving the CPU.
- *In this simplified model.* The Cortex-M4 has no dedicated accumulator: any general-purpose register can play this role.

© Muním Zabidi

Eight parts, two sides: four face memory, four compute.

Part	Side	Job
PC	Faces memory	Address of the next instruction
IR	Faces memory	The instruction just fetched
AR	Faces memory	Address of the data being accessed
DR	Faces memory	Data in transit to or from memory
Decoder	Computes	Works out what the instruction means
CU	Computes	Sequences the steps with control signals
ALU	Computes	Arithmetic and logic
ACC	Computes	Operand and result of the ALU

The $z = x + y$ trace on the next slides puts every one of these to work. Chapter 2 replaces this generic register set with the Cortex-M4's actual sixteen registers.

© Munim Zabidi

Table of Contents

1 Why This Matters

2 Embedded Systems

3 Microprocessor vs Microcontroller

4 Buses and Memory-Mapped I/O

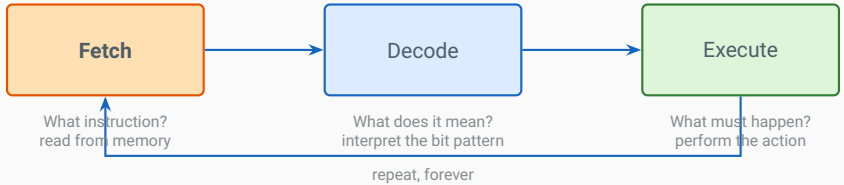
5 Inside the CPU

6 **Execution Cycle and Program Trace**

7 Meet the Board

8 From C to a Pin

The CPU has fetched an instruction. What does it actually do with it?



This is a finite state machine, driven by the clock. You built FSMs in digital electronics. A CPU is the largest one you have met so far. The loop never stops while the machine has power. This is why an embedded program's `main()` is an infinite loop *by design*.

© Muním Zabidi

Consider the simplest possible program:

```
z = x + y;
```

Assume the compiler places `x` at `0x2000`, `y` at `0x2002`, and `z` at `0x2004`.

One line of C becomes three instructions. `LOAD`/`ADD`/`STORE` are plain-English teaching instructions that keep the focus on the flow of data. The real Cortex-M4 ones come at the end.

Program memory

Address	Instruction
0x1000	LOAD 0x2000
0x1002	ADD 0x2002
0x1004	STORE 0x2004

Data memory

Address	Variable	Value
0x2000	x	1
0x2002	y	2
0x2004	z	?

© Munim Zabidi

Six Questions, Six Answers

Each new concept earns its place here by answering one question in this trace:

	Question	Answer
1	What is x?	value at `0x2000`
2	How does the CPU obtain it?	`LOAD`
3	Where does it go?	`ACC`
4	How does addition happen?	the `ALU`
5	Where does the result go?	`ACC`
6	How does it become z?	`STORE`

The next three slides execute this table, step by step

Each question above becomes one phase of fetch–decode–execute. Watch each register earn its answer.

© Muním Zabidi

Step 1 of fetch–decode–execute reads a “machine code instruction.” What is that, really?

Machine language

The lowest level. A bit pattern, such as `0001 1011`, that the Decoder is built to interpret. This is the only language the CPU itself understands.

Assembly language

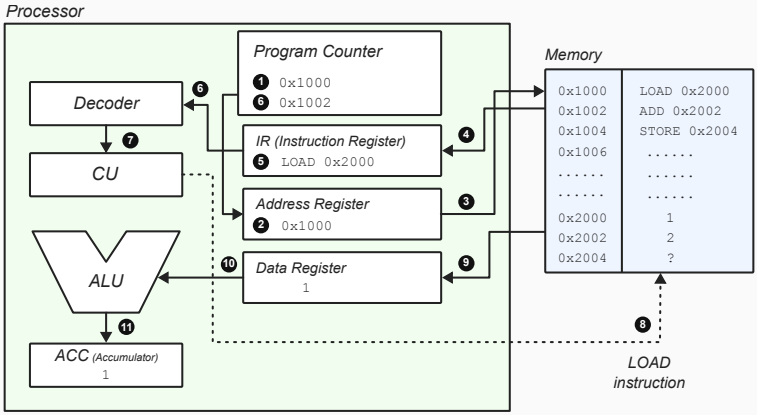
A human-readable mnemonic per instruction — `MOV`, `ADD`, `MUL` — translated to machine code by an **assembler**. One line in, one instruction out.

A **cross assembler** runs on one computer (your PC) and generates machine code for a different one (the Cortex-M4) — the normal situation in embedded development.

Chapter 5 returns to assembly language in depth, once the Cortex-M4’s own registers and instructions are in place.

© Muním Zabidi

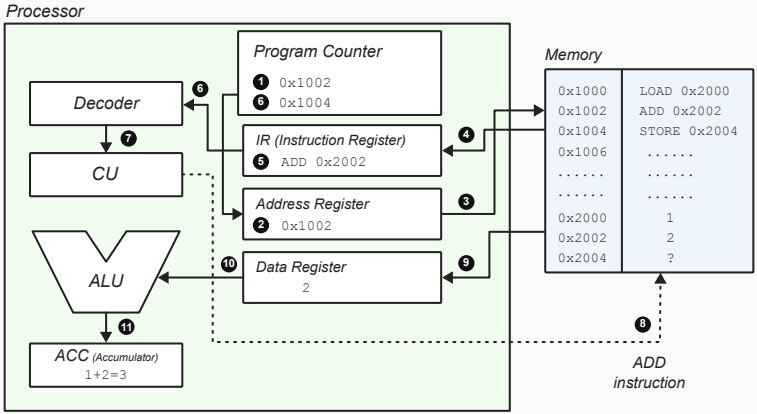
Executing LOAD 0x2000



The value `1` at address `0x2000` reaches the Accumulator. This answers questions 1–3 of the trace. Note step **6**: the PC increases during decode, not after execute.

State: PC `0x1000` → `0x1002` ACC `?` → `1`

Executing ADD 0x2002

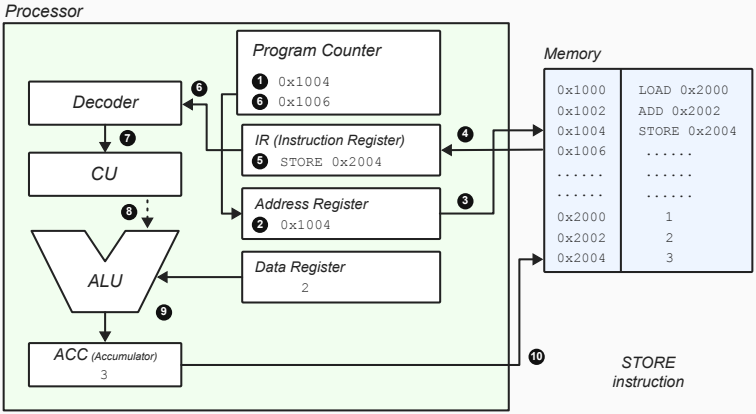


The value 2 is fetched, and the ALU adds it to the Accumulator. The Accumulator becomes 3. This answers questions 4 and 5.

State: PC `0x1002` → `0x1004` ACC 1 → 3

Check yourself: ACC holds 3. What must happen for it to appear at `0x2004`?

Executing STORE 0x2004



The Accumulator's 3 is written out to `0x2004`. This answers question 6. The program is now complete: three instructions, each made of several internal operations.

State: PC `0x1004` → `0x1006` ACC stays 3 memory[`0x2004`] = 3

One Line, Three Levels: C, Assembly, Machine Code

The reveal: this is what the compiler really generates. The data flow is exactly the same.

C: `z = x + y;`

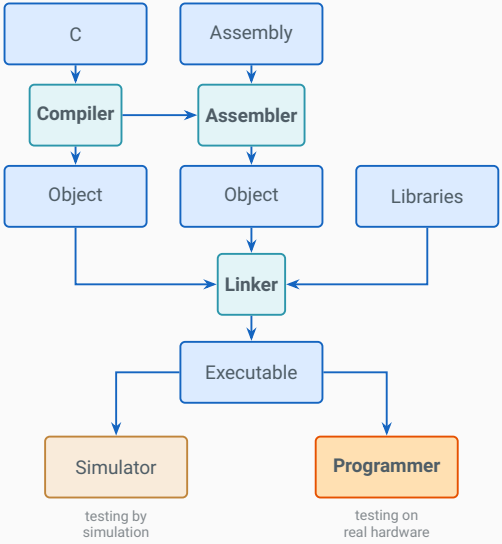
↓ the compiler turns C into assembly; the assembler turns assembly into machine code

Teaching	Assembly	Machine code	Binary
`LOAD x`	`LDR R0, [R1]`	`0x6808`	`0110 1000 0000 1000`
`LOAD y`	`LDR R2, [R3]`	`0x681A`	`0110 1000 0001 1010`
`ADD y`	`ADD R0, R0, R2`	`0x4410`	`0100 0100 0001 0000`
`STORE z`	`STR R0, [R4]`	`0x6020`	`0110 0000 0010 0000`

R1, R3, R4 hold the addresses of x, y, z. The Cortex-M4 adds only registers, so y needs its own load: one C line, four 16-bit machine instructions. Chapters 5–7 explain each one.

© Munim Zabidi

From Source to Executable



Compiler C → assembly or object code. A cross-compiler runs on the host but emits Cortex-M4 instructions.

Assembler assembly → machine code (the table on the previous slide).

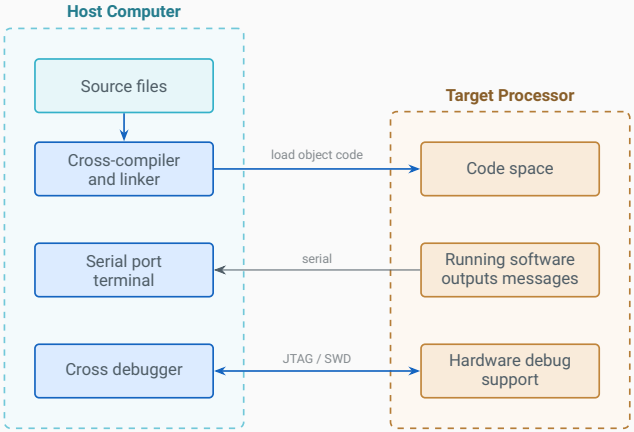
Linker combines objects and libraries and assigns each piece an address in Flash or RAM.

Executable runs in a simulator or is loaded onto real hardware by a programmer.

An IDE bundles these steps behind one button, but they are still separate stages.

© Muním Zabidi

Two Computers: Host and Target



You write and build code on the host (your laptop). It runs on the target (the Cortex-M4), which speaks a different machine language. Three links connect them:

1. **Programming**: object code goes into the target's Flash.
2. **Serial**: the running program reports back to a terminal.
3. **Debug** (JTAG/SWD): halt the CPU and inspect it.

On a Nucleo board the ST-LINK provides all three over one USB cable.

© Muním Zabidi

Table of Contents

1 Why This Matters

2 Embedded Systems

3 Microprocessor vs Microcontroller

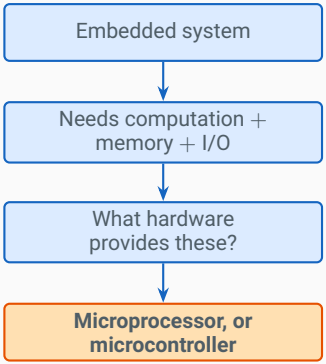
4 Buses and Memory-Mapped I/O

5 Inside the CPU

6 Execution Cycle and Program Trace

7 **Meet the Board**

8 From C to a Pin



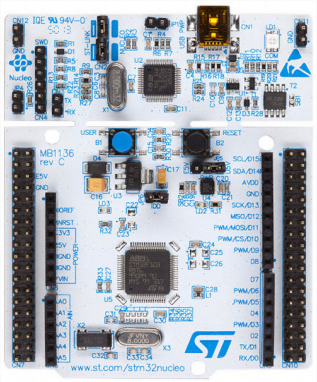
An embedded system is a *requirement*. Something must satisfy that requirement.

The question this raises

What kind of computer belongs inside an embedded system?

We will answer this question using one real board, so every idea in this chapter has a physical home.

© Muním Zabidi



STM32 Nucleo-F446RE

- **ST-LINK** debugger — the upper third of the board, past the perforation. One USB cable handles power, programming, debugging, and a virtual COM port.
- **USER button** (B1) on PC13
- **User LED** (LD2) on PA5

The inner headers follow the Arduino shield layout. The outer **morpho** headers bring out every remaining pin.

Every blink program in this book drives PA5. Everything we discuss from here on will eventually explain this board.

© Munim Zabidi

Table of Contents

1 Why This Matters

2 Embedded Systems

3 Microprocessor vs Microcontroller

4 Buses and Memory-Mapped I/O

5 Inside the CPU

6 Execution Cycle and Program Trace

7 Meet the Board

8 From C to a Pin

We Can Now Answer the Question We Started With

How does a program cause hardware to do something?

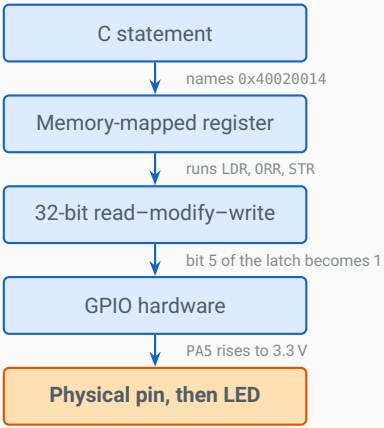
We just traced `z = x + y;` through LOAD, ADD, and STORE — ordinary memory addresses, nothing more. But the same load/store mechanism works on any address. Give it the address of a peripheral register instead of a memory cell, and it changes hardware, not a variable.

Look at this line. It appears in almost every STM32 program ever written:

```
GPIOA->ODR |= (1U << 5);
```

You already know every symbol here: a pointer, a structure member, a compound bitwise-OR assignment, and a left shift.

What may not be obvious is that running this one statement turns on an LED: the LED on PA5, on the board you met at the start of this chapter.



© Muním Zabidi

The chain from the first slides, now with every stage explained:



Bus	Carries	In our chain
Address bus	`0x40020014`	the address of `GPIOA->ODR`
Control bus	`READ` / `WRITE`	`LDR` reads the register, `STR` writes it back
Data bus	the register's bits	`ODR` before, then with bit 5 set
ALU	`ORR`	sets bit 5 between the load and the store

Turning on an LED is writing a number to an address.

Check yourself: why can the CPU turn on an LED without an instruction called `TURN_ON_LED`?

© Muním Zabidi

Summary and What's Next

You can now explain every stage of the chain on the previous slide.

Concepts covered

- Embedded systems and SWaP-C
- Microprocessor vs. microcontroller
- Address, data and control buses
- CPU registers, decoder, CU, ALU
- Fetch–decode–execute
- Memory-mapped I/O

The one takeaway

Turning on an LED is writing a number to an address.

Next: Chapter 2

- The generic CPU becomes the specific Cortex-M4.
- Its real sixteen registers, memory map, and modes.